



CREATE PATTERNS

```
digits = r'\d+'          -- preferred simple format
words  = regex.new([[p{L}+]], regex.ICASE)  -- format for additional options
name   = 'report[1].txt'
exact  = regex.new('^' .. regex.escape(name) .. '$') -- make user strings safe
exact.test('report[1].txt')          -- true
```

`r'\d+'` is equivalent to `regex.new([[d+]])`. In an ordinary quoted string, double regex backslashes: `regex.new('\\d+')`.

Invalid patterns raise `ERR_Syntax`. Wrap dynamic or user-supplied patterns in `try`.

CHOOSE AN INTERFACE

INTERFACE	RESULT / PURPOSE
<code>regex.new(pattern, [flags])</code>	Compile a dynamic pattern
<code>regex.escape(text)</code>	Quote arbitrary text for a pattern
<code>rx.test(text, [flags])</code>	Boolean: any match exists
<code>rx.findFirst(text, [offset], [flags])</code>	First start, exclusive stop, captures
<code>rx.findAll(text, [offset], [flags])</code>	Iterator of positions and captures
<code>rx.extract(text, [offset], [flags])</code>	First match's captures as results
<code>rx.match(text, [flags])</code>	Whole-string match array, or <code>nil</code>
<code>rx.search(text, [flags])</code>	Array of all match arrays, or <code>nil</code>
<code>rx.replace(text, replacement, [flags])</code>	Substituted string
<code>rx.split(text, [flags])</code>	Array of parts between matches

METHOD EXAMPLES

```
has_num = r'\d+'.test('Order 42')          -- true

text = 'Order 42 ready'
start, stop = r'\d+'.findFirst(text)        -- 6, 8
number = text.sub(start, stop)              -- '42'

for start, stop in r'\d+'.findAll('x=10, y=20') do
  print(f'[{start}..{stop}]')
end

name, id = r'(\w+)-(\d+)'.extract('item-42') -- 'item', '42'
date = r'(\d{4})-(\d{2})-(\d{2})'.match('2026-09-10')
year = date?[1]                             -- '2026'
all = r'<(\w+)>'.search('<red> <blue>')
first = all?[0][0]                           -- '<red>'
```

`findFirst()` and `findAll()` return zero-based byte positions as half-open spans `[start, stop)`. Capture arrays are also zero-based.

RETURN CONVENTIONS

<code>findFirst()</code> / <code>findAll()</code>	Capture array excludes the full match; <code>nil</code> when no groups exist
<code>extract()</code>	No results when unmatched; optional unmatched captures are empty strings
<code>match()</code> / <code>search()</code>	Index 0 is the full match; later indices are captures; unmatched calls return <code>nil</code>

CHARACTERS & ESCAPING

PATTERN	MEANING
<code>abc</code>	Literal characters
<code>.</code>	Any code point except a line terminator; <code>DOT_ALL</code> includes them
<code>\t \n \v \f \r</code>	Tab, line feed, vertical tab, form feed, carriage return
<code>\0</code>	NULL (U+0000); other octal escapes are unsupported
<code>\cX</code>	Control character; X is A-Z or a-z
<code>\xHH</code>	Two-digit hexadecimal character
<code>\uHHHH</code>	Four-digit Unicode code point
<code>\u{H...}</code>	Unicode code point up to U+10FFFF
<code>\. * \+</code>	Escaped metacharacter, matched literally

Outside a class, escape `\ ^ $ . | ? * + ( ) [ ] { }` when literal. `regex.escape()` quotes those characters and `-` in dynamic text.

CHARACTER CLASSES

PATTERN	MEANING
<code>[abc]</code>	One of a, b or c
<code>[^abc]</code>	Any code point except those listed
<code>[a-z]</code>	Inclusive code-point range
<code>[-/] [\-\/]</code>	Literal hyphen or slash
<code>[] / [^]</code>	Empty set / any code point including line terminators
<code>\d / \D</code>	ASCII digit / non-digit
<code>\s / \S</code>	Unicode whitespace / non-whitespace
<code>\w / \W</code>	ASCII letter, digit or underscore / complement
<code>\p{...} / \P{...}</code>	Unicode property / property complement

A leading `^` negates a class; elsewhere it is literal. Escape `]` and ambiguous `-`. Most other metacharacters lose their special meaning inside a class.

CHARACTER-CLASS SET OPERATIONS

<code>[\p{sc=Latin}&&\p{LL}]</code>	Intersection: lowercase Latin
<code>[[A-Za-z]--[AEIOUaeiou]]</code>	Subtraction: ASCII consonants
<code>[a-z]q[ch sh th]</code>	Characters or listed strings; longest wins
<code>[\p{sc=Latin}--[a-z]]</code>	Nested class as an operand

Use one operator type at each nesting level. String properties and `\q{...}` sequences cannot be complemented. Escape doubled punctuation used as operators or reserved syntax.

QUANTIFIERS

GREEDY	LAZY	REPETITIONS
<code>*</code>	<code>*?</code>	Zero or more
<code>+</code>	<code>+?</code>	One or more
<code>?</code>	<code>??</code>	Zero or one
<code>{n}</code>	<code>—</code>	Exactly n
<code>{n,}</code>	<code>{n,}?</code>	At least n
<code>{n,m}</code>	<code>{n,m}?</code>	n to m, inclusive

Greedy forms consume as much as possible while still succeeding; lazy forms consume as little as possible. `<.*>` spans to the last `>`; `<.*?>` stops at the first.

ALTERNATIVES, GROUPS & BACKREFERENCES

cat dog	First successful alternative, left to right
(...)	Capturing group; numbered from 1
(?:...)	Non-capturing group
(?<name>...)	Named capture; also receives a number
\1 \2	Text captured by a numbered group
\k<name>	Text captured by a named group

Use (?:...) when grouping only controls precedence or repetition. A quantified capture retains its final iteration; a backreference to a non-participating group matches empty text.

ANCHORS & ZERO-WIDTH ASSERTIONS

^	Start of text; line start with MULTILINE
\$	End of text; line end with MULTILINE
\b / \B	ASCII word boundary / non-boundary
(?=...)	Positive lookahead
(?!...)	Negative lookahead
(?<=...)	Positive lookbehind
(?<!=...)	Negative lookbehind

```
whole_word = r'\bcat\b'           -- not 'concatenate'
bare_amp = r'&(?!amp;|lt;|gt;|#)'
```

Assertions inspect context without consuming text. Inside a class, \b is BEL (U+0008), not a boundary; \B is invalid.

UNICODE PROPERTIES

\p{L}	Any letter	\p{sc=Greek}	Greek script
\p{Lu} / \p{Ll}	Upper-/lowercase letter	\p{scx=Latin}	Latin script extension
\p{N} / \p{Nd}	Any number / decimal digit	\p{White_Space}	Unicode whitespace
\p{P} / \p{S}	Punctuation / symbol	\p{Emoji}	Emoji property
\p{Z} / \p{C}	Separator / other		

Identifier: \p{ID_Start}\p{ID_Continue}*. Long names work too: \p{Script=Latin}. \w remains ASCII-oriented; use properties for international text.

COMPILATION & INLINE FLAGS

regex.ICASE / (?i:...)	Unicode case-insensitive matching
regex.MULTILINE / (?m:...)	^ and \$ recognise line boundaries
regex.DOT_ALL / (?s:...)	. includes line terminators
(?ims:...)/ (?i-s:...)	Enable several / enable i, disable s

```
line = regex.new([!^error:], regex.ICASE | regex.MULTILINE)
local_case = r'ID-(?i:[a-f0-9]{8})'
```

PATTERN OBJECT PROPERTIES

rx.pattern original pattern · rx.flags compilation mask · rx.error always nil for script-accessible objects.

MATCH & REPLACEMENT FLAGS

regex.CONTINUOUS	Match must begin at text position 0
regex.NOT_NULL	Reject zero-length matches
regex.NOT_BEGIN_OF_LINE / NOT_END_OF_LINE	Suppress edge line boundaries
regex.NOT_BEGIN_OF_WORD / NOT_END_OF_WORD	Suppress edge word boundaries
regex.PREV_AVAILABLE	A preceding character is available to assertions
regex.REPLACE_NO_COPY	Emit replacements without unmatched text
regex.REPLACE_FIRST_ONLY	Replace only the first match

Pass these to an operation, not regex.new(). Do not mix compilation and match flags in one mask.

REPLACEMENT PLACEHOLDERS

\$0 or \$&	Full match
\$1, \$2, ...	Numbered capture
\$<name>	Named capture
\$` / \$'	Text before / after the match
\$\$	Literal dollar sign

```
phone = r'(?<area>\d{3})-(?<local>\d{3})-(?<line>\d{4})'
phone.replace('555-123-4567', '($<area>) $<local>-$<line>')
-- '(555) 123-4567'
r'\d+'.replace('1 2 3', 'X', regex.REPLACE_FIRST_ONLY) -- 'X 2 3'
r'\s*'.split('r, g, b') -- {'r','g','b'}
```

COMMON RECIPES

PATTERN / OPERATION	PURPOSE
^[+ -]?d+\$	Integer shape
^[+ -]?d+(?:\.\d+)?	Decimal shape
^d{4}-d{2}-d{2}\$	ISO-style date shape
^\p{ID_Start}\p{ID_Continue}*\$	Unicode identifier
r'^\s+ \s+\$'.replace(text, '')	Trim outer whitespace
r'\s+'.replace(text, '')	Collapse whitespace
\.([A-Za-z0-9]+)\$	Capture a file extension
<([>]+)>(.*?)</\1>	Text inside simple paired tags

Shape checks do not validate domain semantics; the date pattern accepts impossible dates.

BOUNDARIES & PERFORMANCE

- Unsupported: \Q...E, possessive quantifiers, recursion, conditionals, inline comments, POSIX classes, .NET balancing groups and octal escapes other than \0.
- Compile once and reuse. A deferred expression <{ regex.new(...) }> compiles on first use.
- Use test() for presence and findFirst() for positions; avoid collecting results unnecessarily.
- Anchor validation patterns and prefer specific classes over broad .* searches.
- Avoid ambiguous nested quantifiers such as (a+)+; they can cause catastrophic backtracking.